

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

09.03.03 – Прикладная информатика

Профиль: Прикладной искусственный интеллект

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

Тема работы:

‘БЫКИ И КОРОВЫ’

Чупрасова Алёна Владимировна 25948,
Коваленко Александра Андреевна 25948

ОГЛАВЛЕНИЕ

1	ТЕРМИНЫ И СОКРАЩЕНИЯ	3
2	ВВЕДЕНИЕ. КОНЦЕПТ ИГРЫ «БЫКИ И КОРОВЫ».....	3
3	ЦЕЛЬ И ОБЛАСТЬ ПРИМЕНЕНИЯ.....	3
4	РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	4
5	ФУНКЦИОНАЛЬНЫЕ ХАРАКТЕРИСТИКИ.....	4
6	ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	5
6.1	MAIN.....	5
6.1.1.	Внешний вид(Interface).....	5
6.1.2	История	6
6.1.3.	Блок обработки истории.	7
6.1.4.	Блок текущего вывода.....	8
6.1.5.	Блок отрисовки надписей Win/Loser.	9
6.1.6.	Блок чтения с терминала + блокировка повторного ввода одной цифры.	10
6.1.7.	Три основных функциональных модуля.	10
6.2.	CPU_CORE.....	12
6.2.1.	Центральный процессор Cdm-8. Управление состоянием игры.	13
6.2.2.	Порт ввода данных.....	14
6.2.3.	Порт ввода кода генерации.	15
6.2.4.	Порт вывода.	15
6.2.4.	Сигнал подтверждения.	16
6.2.5.	Декодер управления памятью.	16
6.2.6.	Блок входных управляющих сигналов и мультиплексор адресной шины.	17
6.2.7.	Постоянное запоминающее устройство (ПЗУ).....	18
6.2.8.	Оперативно запоминающее устройство (ОЗУ).	18
6.3.	INPUT_IO.....	19
6.3.1.	Клавиатурный ввод и приоритетный шифратор.	19
6.3.2.	Шестнадцатеричные индикаторы.	20
6.3.3.	Мультиплексор ввода данных.	21
6.3.4.	Блок генератора кода.	22
6.4.	GEN_CODE.....	22
6.5.	OUTPUT_IO.....	24
6.5.1.	Дешифратор адреса (Address Decoder).	24
6.5.2.	Учет статистики и индикация результата.....	25
6.5.3.	Детекция нулевого значения.....	26
6.5.4.	Визуальная индикация игровой статистики.	26
6.6.	bcd.....	27
6.7.	block.....	27
6.8.	assembler.....	28
7	ЗАКЛЮЧЕНИЕ	30
8	ИСТОЧНИКИ.....	30

1 ТЕРМИНЫ И СОКРАЩЕНИЯ

Термин / Сокращение	Описание
Logisim	Цифровая среда для построения логических схем.
ПЗУ	Постоянно запоминающее устройство.
ЦПУ	Центральное процессорное устройство. Электронный блок либо интегральная схема, исполняющая машинные инструкции (код программ). Главная часть аппаратного обеспечения компьютера или программируемого логического контроллера.
CdM – 8	8-битный учебный процессор.
Assembler	Низкоуровневый язык программирования, который представляет машинный код в читаемой для человека форме.
Коровы	Правильно угаданные цифры на неправильных позициях
Быки	Правильно угаданные цифры на правильных позициях
Гарвардская архитектура	Это модель построения компьютера с отдельным хранением команд и данных на физическом уровне.
ОЗУ	Оперативно запоминающее устройство.

2 ВВЕДЕНИЕ. КОНЦЕПТ ИГРЫ «БЫКИ И КОРОВЫ».

В рамках курса «Цифровые платформы 25/26» студентам необходимо разработать интерактивную систему с использованием логических схем и аппаратного моделирования.

Проект «Быки и коровы» представляет собой реализацию одноимённой логической игры средствами цифровой схемотехники в среде моделирования Logisim в связке с учебным процессором CDM-8. В данной игре пользователь должен угадать загаданное число, получая подсказки в виде количества «быков» и «коров». «Быки» обозначают цифры, угаданные на правильных позициях, а «коровы» — цифры, присутствующие в числе, но находящиеся не на своих местах. В сгенерированном коде гарантируется, что цифры разные.

Особенностью можно выделить связку логических схем на платформе Logisim и учебного языка Assembler в соотношении 50/50.

3 ЦЕЛЬ И ОБЛАСТЬ ПРИМЕНЕНИЯ

Целью данной работы является разработка цифровой системы, реализующей игру «Быки и коровы», с использованием логических схем и базовых компонентов цифровой электроники.

В процессе разработки решаются задачи:

- реализация логики сравнения чисел (Assembler);
- подсчёт количества «быков» и «коров»(Assembler);
- организация ввода пользовательских данных (Logisim);
- вывод результатов на устройства отображения(Logisim);
- управление игровым процессом (сброс, новая попытка) (Logisim).

Область применения данной разработки — образовательная. Изучить процессор CDM-8, а также увидеть принцип работы логических схем Logisim.

Проект также представляет интерес для студентов и преподавателей, так как показывает, каким образом классические задачи могут быть реализованы наглядно.

4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Игровой процесс организован следующим образом:

1. Игрок должен включить такты. Частоты 128 герц вполне достаточно.
2. Пользователь вводит число с помощью внешнего устройства ввода (реализована клавиатура). Введённое значение поступает в систему и сохраняется для последующей обработки.
3. Загаданное число генерируется схемой на основе генератора случайных чисел.
4. После завершения ввода данные передаются в процессор CdM-8. Процессор выполняет программу на ассемблере, реализующую алгоритм сравнения двух чисел:
 - определяется количество совпадающих цифр на правильных позициях («быки»);
 - определяется количество совпадающих цифр на неправильных позициях («коровы»).
5. Результаты вычислений передаются обратно в аппаратную часть, где отображаются на устройствах вывода (шестнадцатеричные индикаторы и матрицы).
6. Система отслеживает количество попыток пользователя. Дается 25 попыток.
7. При угадывании числа формируется сигнал завершения игры (на матрицу выводится надпись Win!). В случае проигрыша выводится Loser. Система может быть переведена в исходное состояние по нажатию соответствующей (подписанной) кнопки на игровом поле.

5 ФУНКЦИОНАЛЬНЫЕ ХАРАКТЕРИСТИКИ

Для реализации логики игры «Быки и коровы» используется комбинация аппаратной части, разработанной в среде моделирования цифровых схем (Logisim), и программной части, реализованной на языке ассемблер для учебного процессора CdM-8.

Аппаратная часть отвечает за ввод и вывод данных, хранение чисел, а также за базовые операции сравнения и подсчёта. Процессор CdM-8 используется для реализации алгоритмической логики игры, включая обработку введённого пользователем числа и вычисление количества «быков» и «коров».

Программная часть на процессоре CdM-8 выполняет следующие функции:

- приём и обработка входных данных;
- реализация алгоритма сравнения чисел;
- организация циклов проверки и переходов;
- управление передачей данных между регистрами и памятью;
- формирование управляющих сигналов для аппаратной части.

6 ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

Аппаратная часть проекта представляет собой набор взаимосвязанных цифровых схем, реализованных в среде моделирования (Logisim). Система построена по модульному принципу и включает в себя блоки ввода, обработки, хранения и вывода данных.

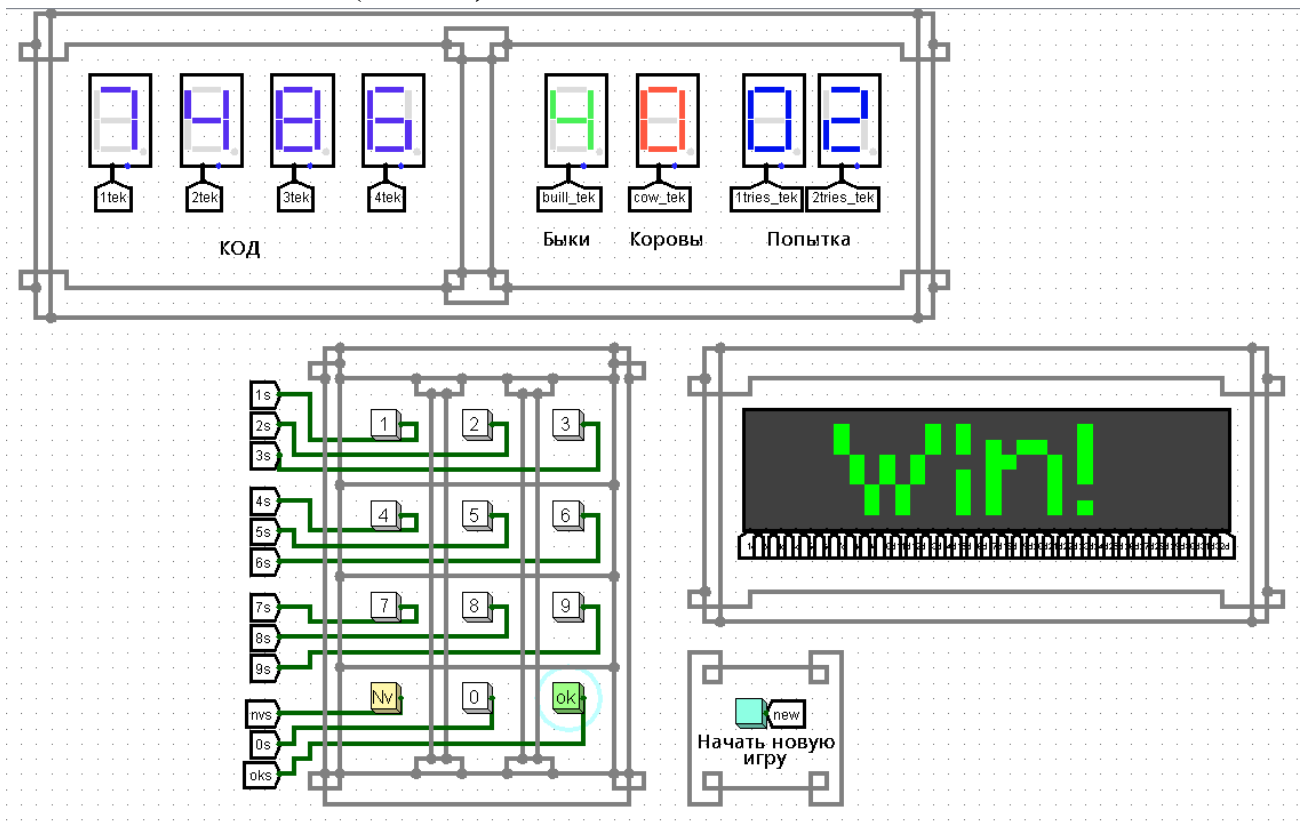
Ключевой особенностью проекта является использование учебного процессора **CdM-8**, который выполняет программную часть игры на языке ассемблер. Аппаратные схемы обеспечивают взаимодействие пользователя с системой, а также реализацию на языке ассемблер.

Ниже представлено описание основных подсхем проекта.

6.1 MAIN

Схема MAIN состоит из множества ключевых частей.

6.1.1. Внешний вид(Interface)

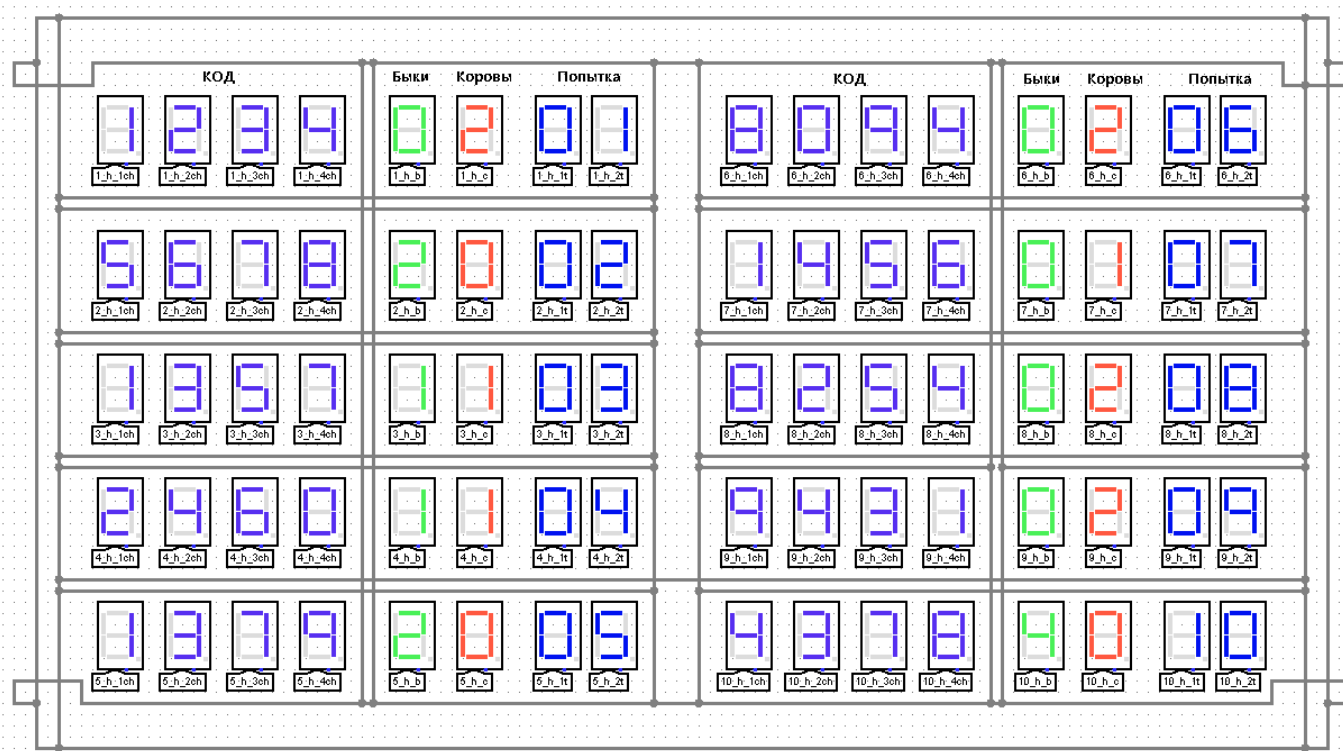


Скриншот 1. Внешний вид.

В данной подсхеме реализован интерфейс игры. Игрок осуществляет взаимодействие именно здесь. Элементы из которых состоит эта подсхема:

1. 8 шестнадцатеричных индикаторов, указывающие на введенный игроком текущий код, быков, коров и попытку.
2. Клавиатура, реализованная как стандартный вид калькулятора для ввода кода.
3. Кнопка «OK» для передачи в обработку кода игрока, готовность записанных чисел.
4. Кнопка «Nv» для сброса введенного ранее кода (так же используется при ошибке пользователя при вводе).
5. Кнопка «New» - начать новую игру.
6. Светодиодная матрица 8x32 для отображения Win/Loser.

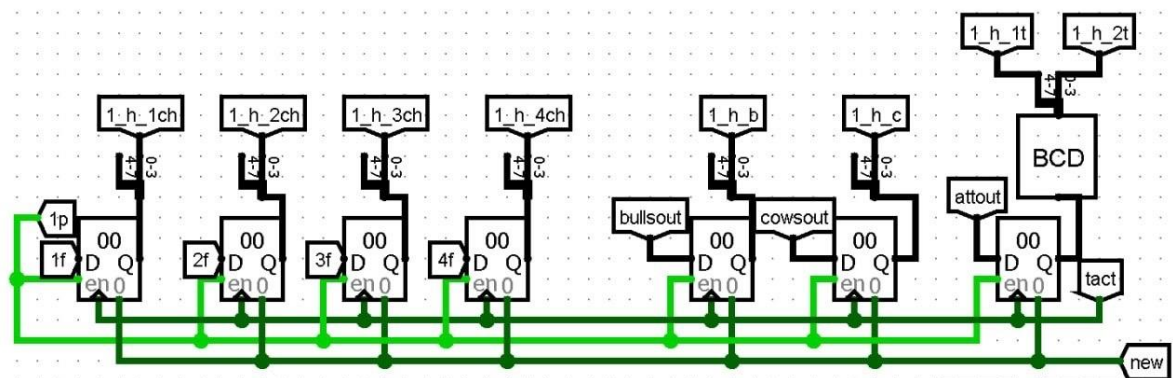
6.1.2 История



Скриншот 2. «История ввода».

Здесь реализована история ранее введенных кодов игроком, а также быки, коровы и попытка для него. В ней отображаются последние 10 попыток для анализа пользователем. Аналогично используются 8 шестнадцатеричных индикаторов.

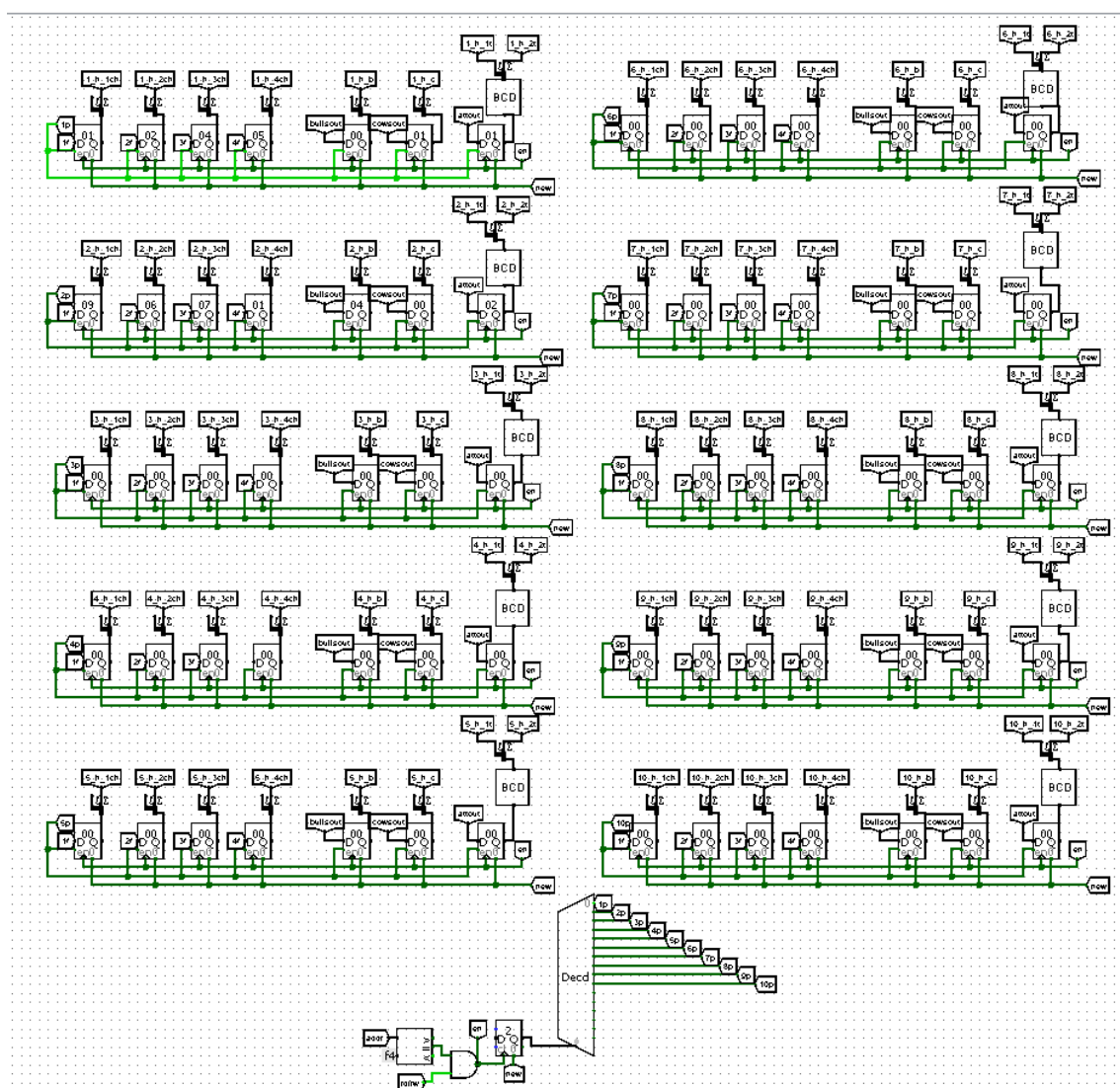
6.1.3. Блок обработки истории.



Скриншот 3. Вывод последнего кода.

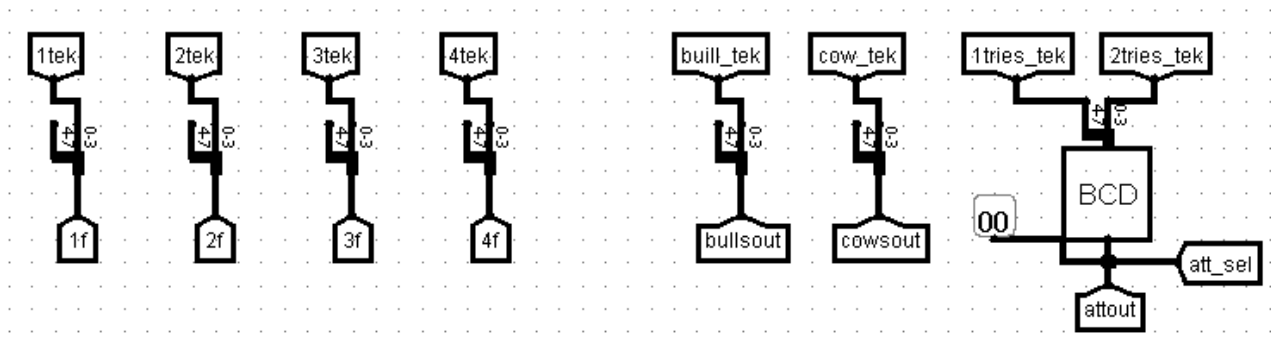
На данном этапе обрабатывается вывод с интерфейса игрока на блок истории последней попытки. Получает сигнал как адрес F4 и сигнал обнуления «ОК» (конец главного цикла).

По аналогии реализовано 10 подобных блоков.



Скриншот 4. 10 блоков истории.

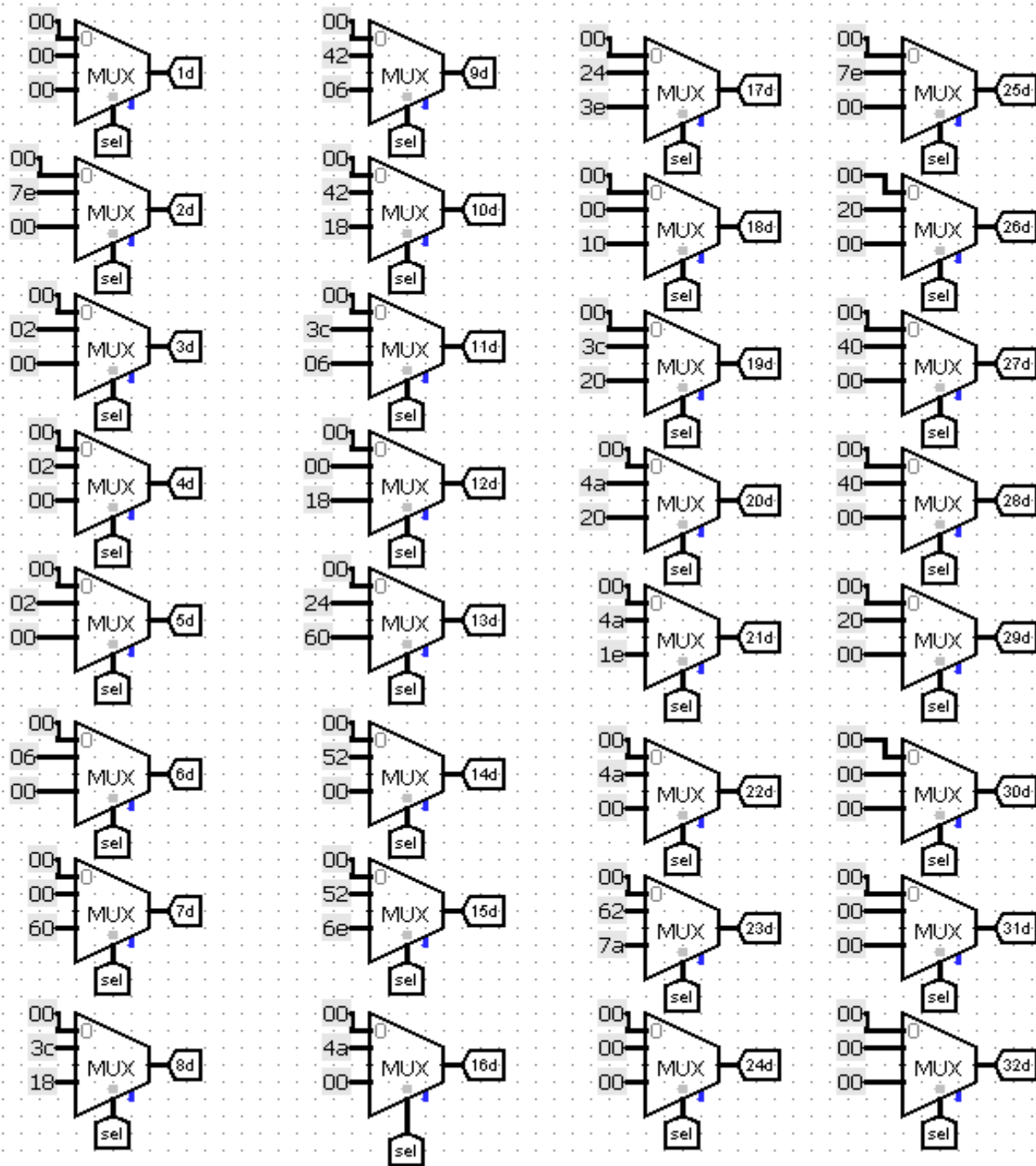
6.1.4. Блок текущего вывода.



Скриншот 5. Блок вывода.

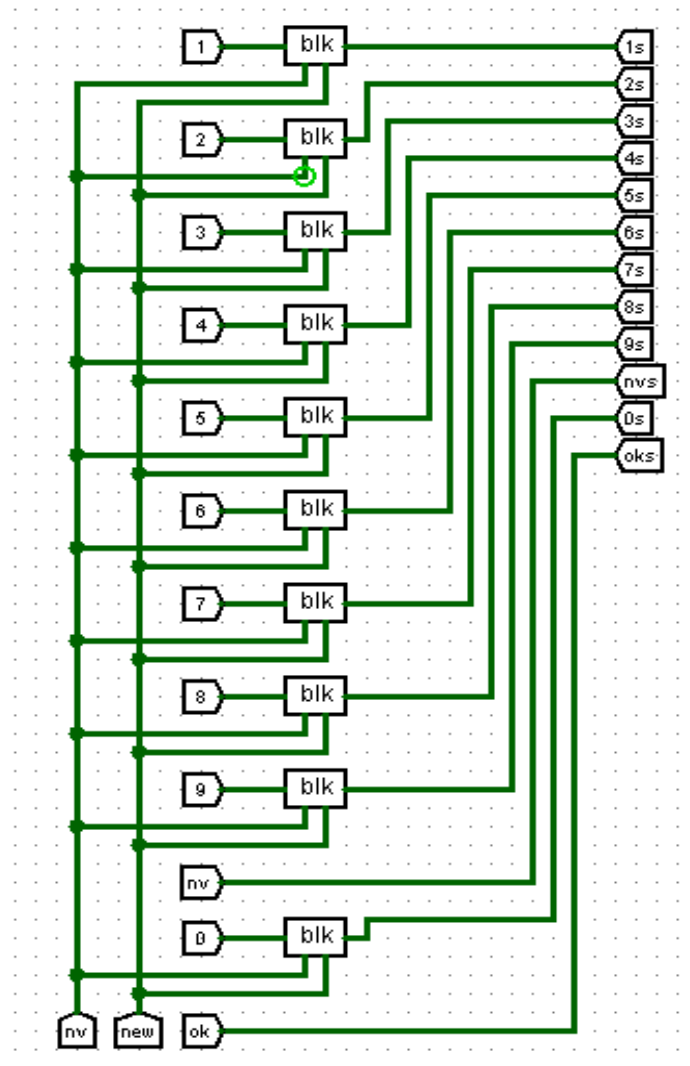
Данный блок отвечает за текущий вывод. Информация поступает с другой подсхемы (см. пункт 4.5.)

6.1.5. Блок отрисовки надписей Win/Loser.



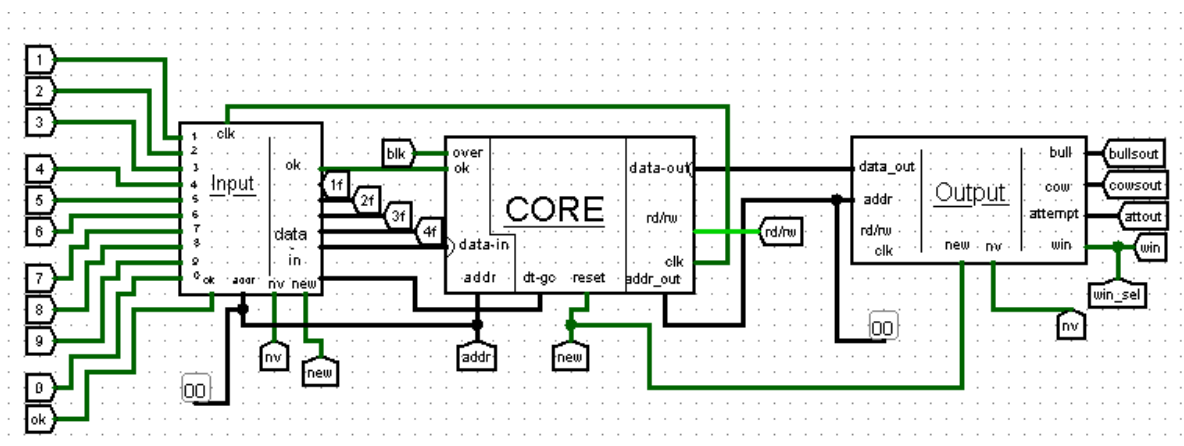
Скриншот 6. Отрисовка надписей Win/Loser.

6.1.6. Блок чтения с терминала + блокировка повторного ввода одной цифры.



Скриншот 7. Чтение + блокировка.

6.1.7. Три основных функциональных модуля.



Скриншот 8. «Мозг» компьютера.

Схема демонстрирует архитектуру системы, разделенную на три основных функциональных модуля, соединенных шинами данных и управления.

1. Модуль ввода (Input) содержит логику записи цифр и сравнения адресов.

Входы:

- 1–9, 0: Дискретные входы кнопок клавиатуры (цифры).
- ok: Сигнал подтверждения ввода.

Выходы:

- 1f, 2f, 3f, 4f: Сигналы, передающие значения четырех разрядов введенного числа в процессор.
- data_in: Шина данных ввода.
- addr: Шина адреса (формируется на основе нажатых кнопок).
- nv, new: Сигналы управления состоянием (NV / New Game).

2. Центральный процессор (CORE) исполняет программы игры, логику сравнения и управляет периферией.

Входы:

- data_in: Данные, полученные от модуля Input.
- 1f, 2f, 3f, 4f: Для считывания разрядов введенного числа.
- addr, reset, clk: Системные сигналы (адрес, готовность данных, сброс, тактирование).

Выходы:

- data-out: Шина данных вывода (результаты вычислений, команды для дисплея).
- rd/rw: Сигнал чтения/записи.
- addr: Адресная шина (процессор обращается к памяти или портам ввода-вывода).
- over, ok: Сигналы статуса выполнения.

3. Модуль вывода и состояния (Output)

Входы:

- data_out: Данные от процессора (например, количество быков/коров).
- addr, rd/rw, clk: Сигналы синхронизации и управления шиной.
- new, nv: Сигналы сброса и нового ввода.

Выходы (Счетчики и Флаг):

- bull: Счетчик «Быков» (верная цифра на верном месте).
- cow: Счетчик «Коров» (верная цифра на неверном месте).
- attempt: Счетчик попыток.
- win: Флаг победы (генерируется, когда bull = 4).

4. Глобальные связи (Шины)

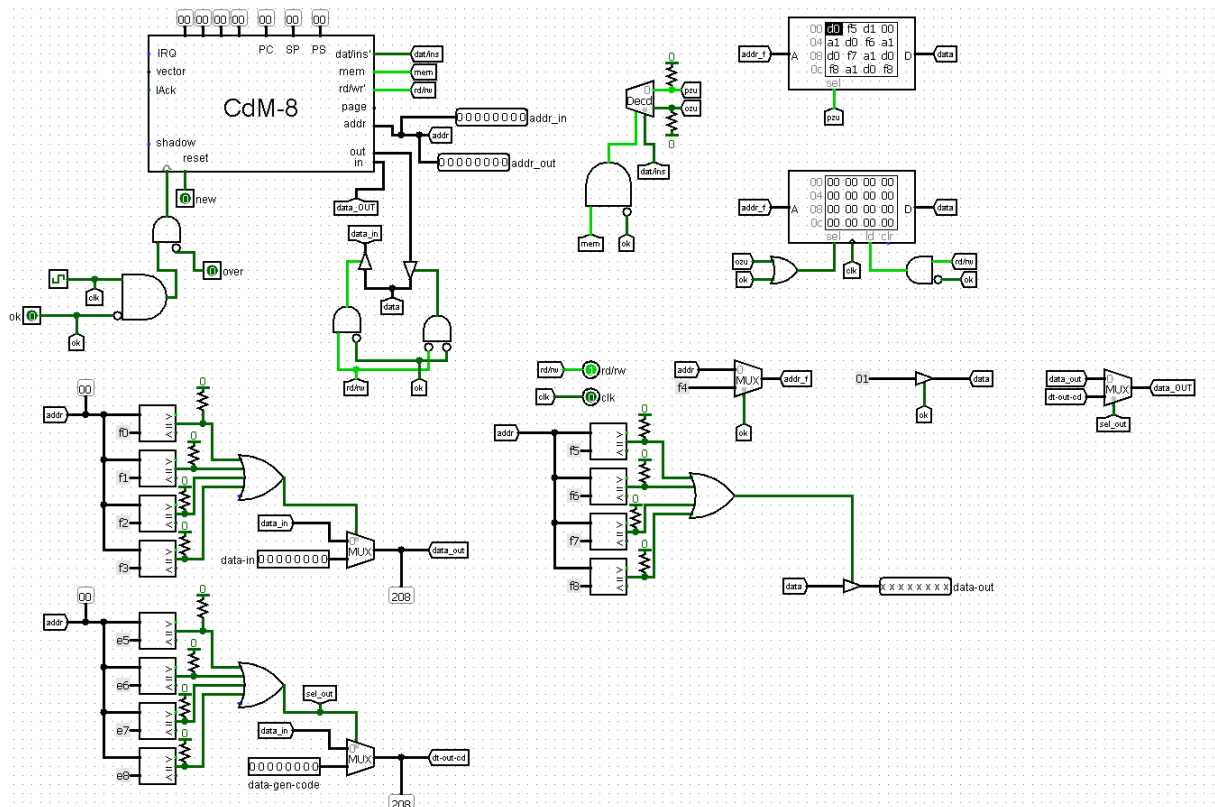
Шина данных (data): Связывает Input - CORE - Output. Процессор читает ввод из Input и пишет результаты в Output.

Шина адреса (addr): Используется процессором для адресации регистров блоках Input и Output.

Сигнал rd/nw: Глобальный строб чтения/записи, синхронизирующий обмен данными между процессором и периферией.

Сигнал ok: Синхронизирует шаг игры (нажатие кнопки пользователем).

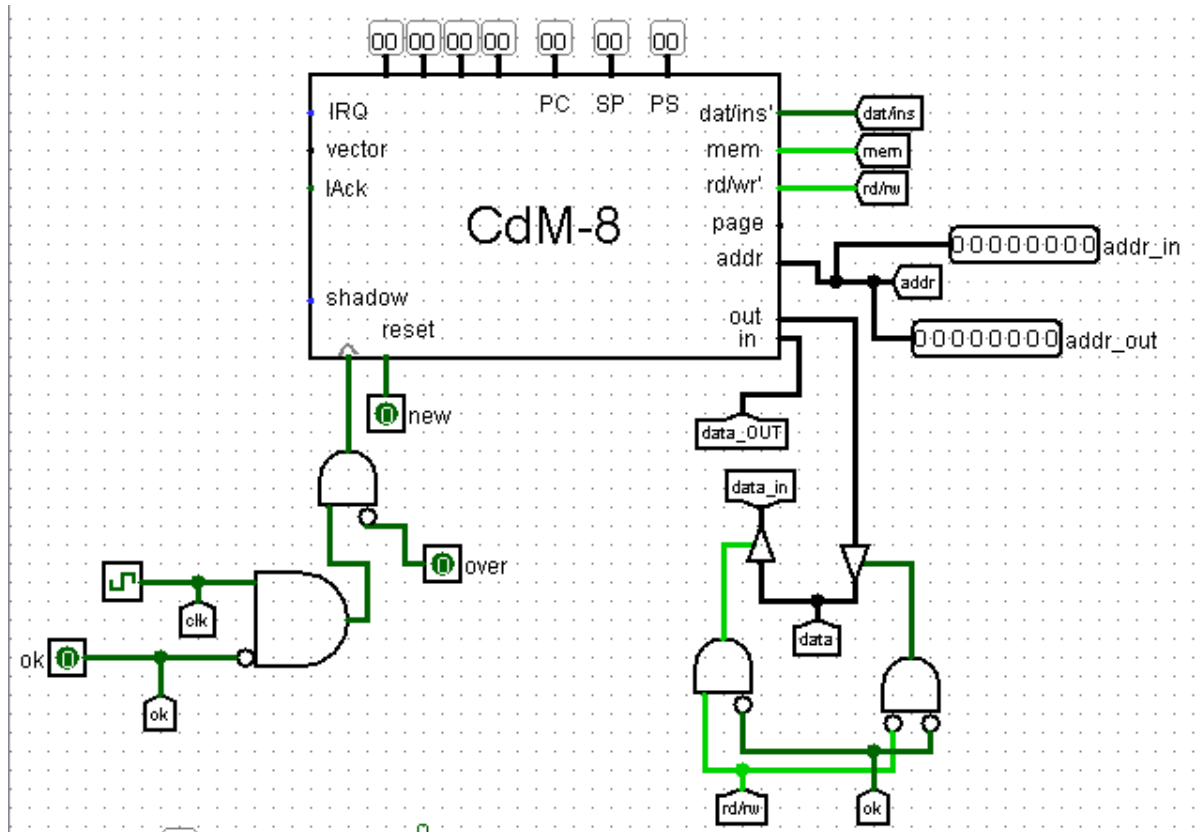
6.2. CPU_CORE.



Скриншот 9. CPU_CORE

На схеме реализована система на базе процессора **CdM-8**, использующая принцип отображения ввода-вывода на память (Memory-Mapped I/O). Процессор управляет игрой, обращаясь по специфическим адресам к блокам ввода данных игрока, генерации секретного кода и вывода результатов проверки. Система дешифрации адресов (группы компараторов) распознает обращения процессора и через мультиплексоры направляет данные в нужные регистры. Логические вентили и элементы памяти обеспечивают передачу состояния игры, синхронизацию ходов и отображение количества «быков» и «коров».

6.2.1. Центральный процессор Cdm-8. Управление состоянием игры.

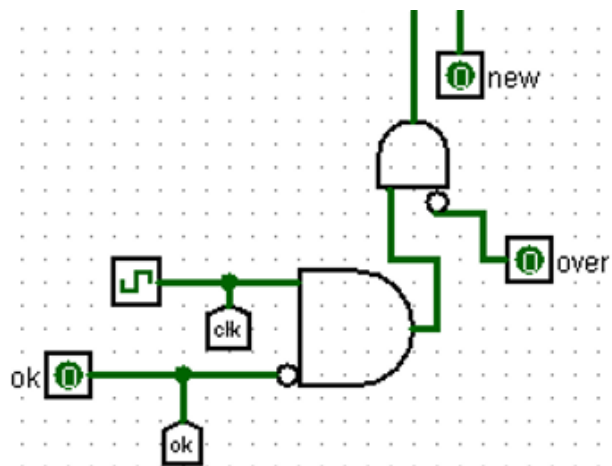


Скриншот 10. Процессор Cdm-8.

«Мозг» системы. Это процессор, который выполняет ассемблерный код игры.

Основные пины:

- addr: Шина адреса. Процессор выдает сюда число, чтобы сказать, с каким устройством он хочет взаимодействовать.
- data/ins: Сигнал данных или инструкций.
- out / in: Управление вводом/выводом данных.
- rd/wr: Сигнал чтения/записи.
- reset: Сброс процессора.



Скриншот 11. Управление состоянием.

Элементы: Входы ok, clk, логические элементы (И), сигналы new, over.

- ok: Кнопка «ОК».
- clk: Тактовый сигнал.

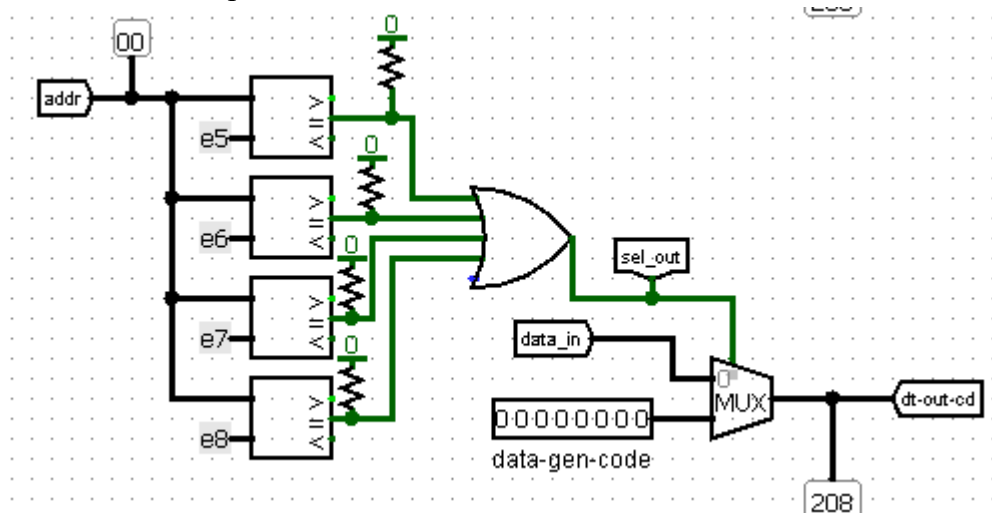
- Логика: Схема из вентилях (AND) обрабатывает прерывание на «ОК» для безопасной работы шины данных и сигналы сброса.

Скриншот 12. Схема ввода цифр.

- Дешифратор адреса (компараторы f0–f3):
Осуществляет сравнение входного адреса (addr) с предустановленными значениями. В случае совпадения адреса с одним из диапазонов f0–f3 передаются данные.

- Логический элемент ИЛИ:
Выполняет объединение выходов компараторов, формируя сигнал выбора устройства. При обращении к любому из адресов f0–f3 на выходе формируется логическая «1».
- Мультиплексор (MUX):
Управляется сигналом с выхода элемента ИЛИ. Реализует следующую логику:
 - При активном сигнале выбора (sel=1) — передаёт внешние данные (data_in) на выход data_out;
 - При неактивном сигнале (sel=0) — передаёт внутренние данные.
- Резисторы (pull-up):
Предотвращают неопределённые состояния.

6.2.3. Порт ввода кода генерации.



Скриншот 13. Схема записи.

Реализация порта ввода для считывания сгенерированного игрового кода процессором.

Состав:

Дешифратор адреса (компараторы e5–e8): Сравнивает адресную шину с заданными значениями. При совпадении формируется сигнал выбора устройства.

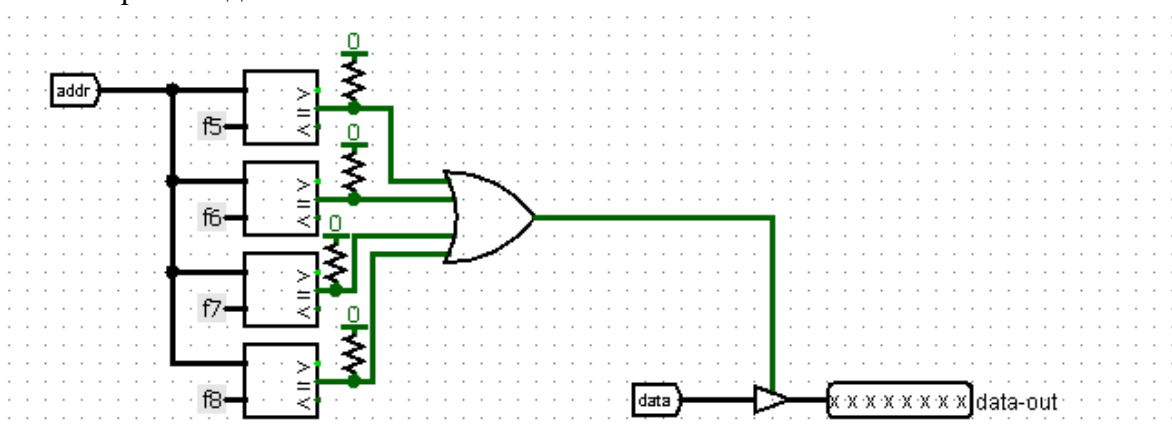
Логический элемент ИЛИ: Объединяет выходы компараторов, формируя управляющий сигнал sel_out.

Мультиплексор (MUX):

- При активном сигнале sel_out пропускает данные с входа data_in (где формируется код через блок data-gen-code) на выход dt-out-cd.
- При неактивном сигнале передает внутренние данные.

Принцип действия: Обеспечивает изолированную передачу сгенерированного кода на шину данных только при обращении процессора к адресам e5–e8.

6.2.4. Порт вывода.



Скриншот 14. Вывод.

Реализация порта вывода для отображения результатов проверки (количества быков и коров), а также количество попыток и флага победы. Фиксирует и визуализирует данные, записанные процессором в диапазоны адресов f5–f8.

Состав:

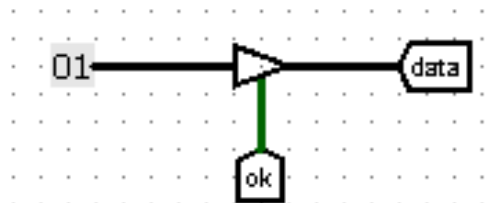
Дешифратор адреса (компараторы f5–f8): Детектирует обращение процессора к адресам вывода.

Логический элемент ИЛИ: Формирует сигнал разрешения записи при совпадении адреса.

Буфер (Buffer): Управляется сигналом с элемента ИЛИ.

- При активации пропускает данные с шины data на выходной индикатор data-out.
- В неактивном состоянии ничего не передаёт.

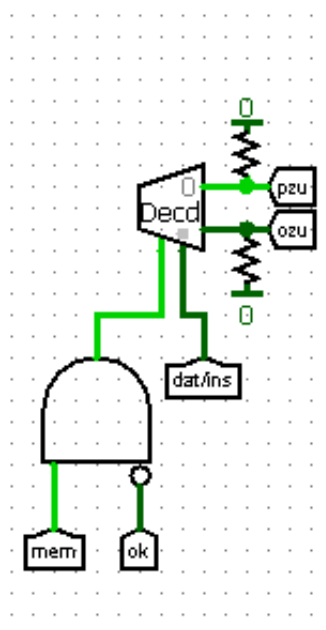
6.2.4. Сигнал подтверждения.



Скриншот 15. Генератор сигнала подтверждения.

Формирование флага подтверждения или управляющего сигнала. Константа «1» - источник фиксированного логического значения. Управляемый буфер пропускает сигнал «1» только при активном флаге «ok».

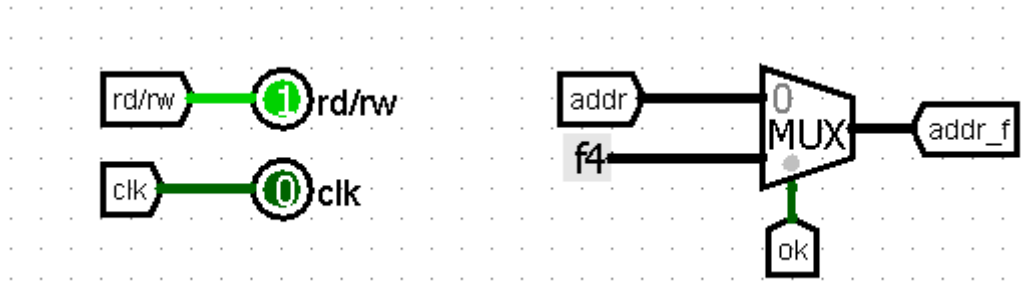
6.2.5. Декодер управления памятью.



Скриншот 16. Декодер.

Формирование сигналов выбора блоков памяти и управления доступом. Декодер принимает входные сигналы («mem») и подтверждение («ok»). Выходы «psu» и «csu» - сигналы выбор ПЗУ или ОЗУ.

6.2.6. Блок входных управляющих сигналов и мультиплексор адресной шины.



Скриншот 17. Блок входных управляющих сигналов (слева) и мультиплексор адресной шины для фиксации флага ok (справа)

Подача глобальных сигналов управления состоянием системы.

Блок выходных управляющих сигналов (слева):

- Сигнал rd/rw (Read/Write): Сигнал чтения/записи.
- Сигнал clk (Clock): Передача тактов.

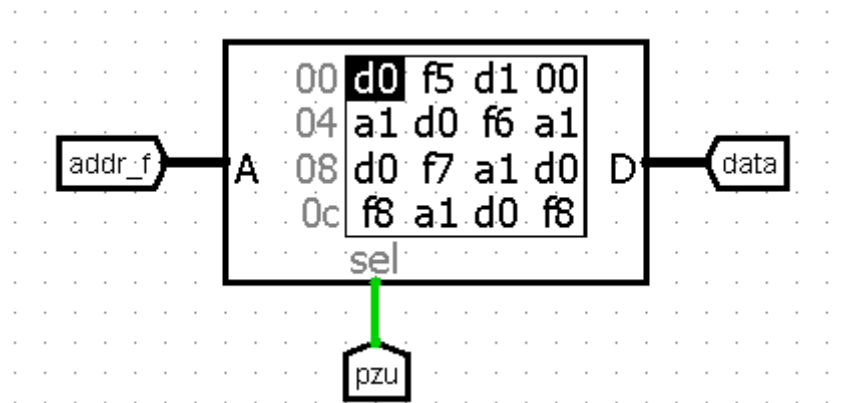
Мультиплексор адресной шины (справа):

- Мультиплексор (MUX): Устройство выбора одного из двух входных адресов для передачи на выход addr_f.
- Входы данных:
 - Вход 0: Подключен к основной шине адреса процессора (addr).
 - Вход 1: Подключен к константе f4.
- Управляющий вход (sel): Подключен к сигналу ok

Принцип действия:

- Если сигнал ok не активен, на выход addr_f передается обычный адрес процессора (addr).
- Если сигнал ok активен, мультиплексор переключается и принудительно выдает адрес f4. Это позволяет системе игнорировать текущий адрес процессора и обращаться к конкретному блоку.

6.2.7. Постоянное запоминающее устройство (ПЗУ).



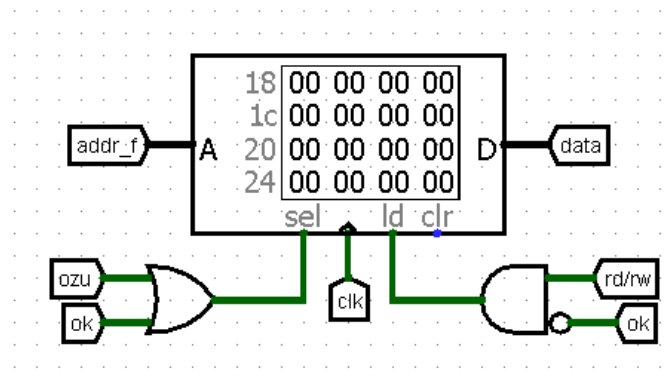
Скриншот 18. ПЗУ.

Содержит в себе инструкции Assembler (образ памяти).

Состав и параметры:

- Адресный вход (A): Подключен к шине addr_f.
- Выход данных (D): Подключен к шине data.
- Сигнал выбора (sel): Управляется внешним сигналом pzu (активация ПЗУ).

6.2.8. Оперативно запоминающее устройство (ОЗУ).



Скриншот 19. ОЗУ.

Хранение переменных данных игры (текущее состояние, счетчики, промежуточные результаты).

Состав и параметры:

Адресный вход (A): Подключен к шине addr_f.

Порт данных (D): Подключен к шине data (двунаправленный или вход/выход в зависимости от режима).

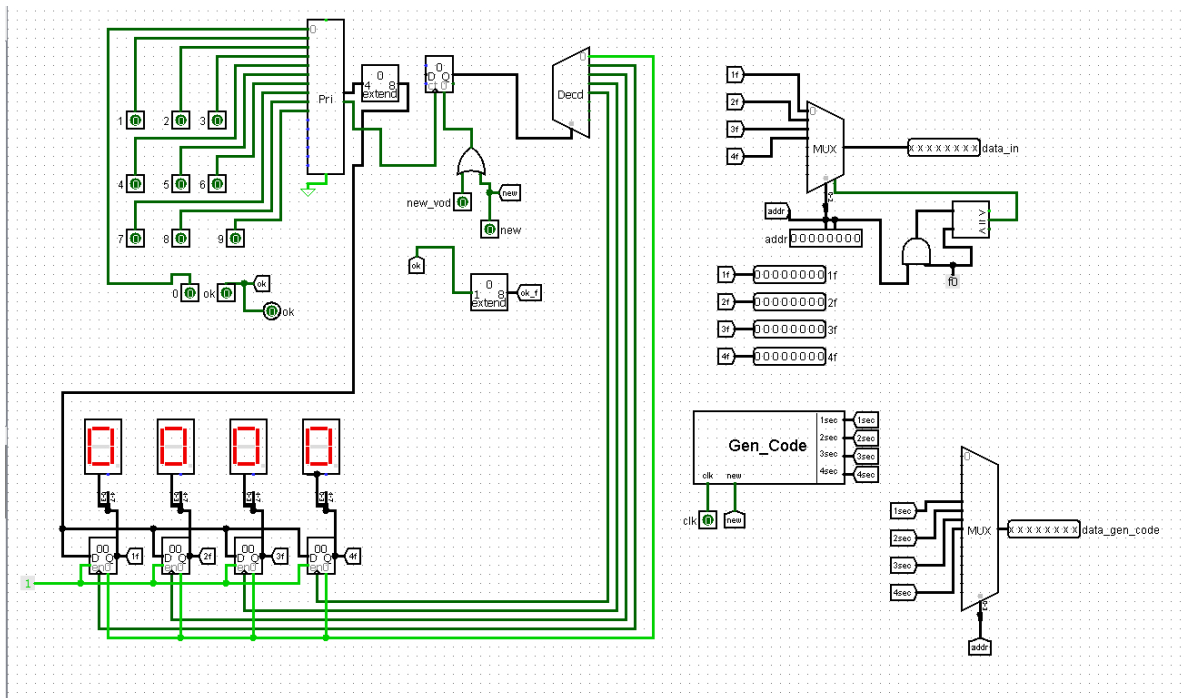
Логика выбора (sel):

- Реализована через элемент ИЛИ.
- Активируется при поступлении сигнала ozu (активация ОЗУ) ИЛИ сигнала ok (подтверждение действия).

Синхронизация записи (ld):

Подключен к тактовому сигналу clk. Сигнал ok как прерывание.

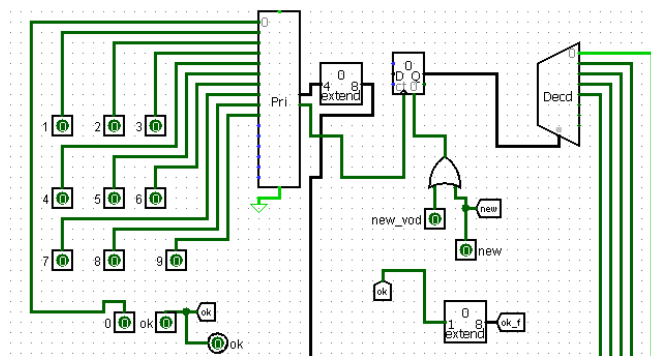
6.3. INPUT_IO.



Скриншот 20. Схема «INPUT_IO».

Представленная схема реализует подсистему ввода. Схема обеспечивает ввод цифр пользователем, отображение информации на 4 шестнадцатеричных индикаторах и маршрутизацию данных между различными блоками системы по сверке адресов через мультиплексор.

6.3.1. Клавиатурный ввод и приоритетный шифратор.



Скриншот 21. Клавиатура и шифратор.

Преобразование нажатий кнопок (0-9) в двоичный код.

Состав и функционирование:

Приоритетный шифратор (Pri): Принимает сигналы от 10 кнопок (цифры 0-9).

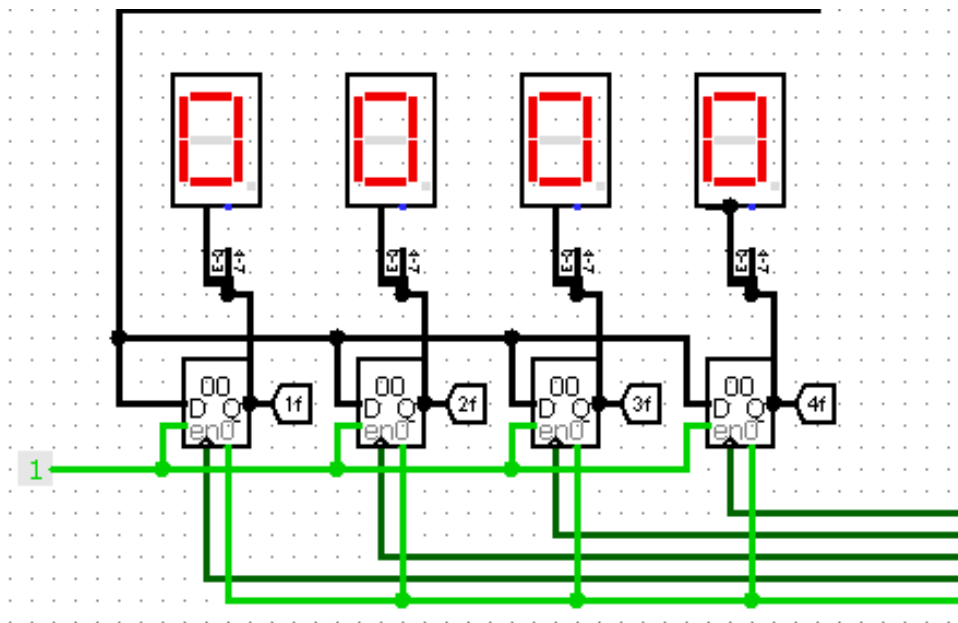
Расширители разрядности (Extend): Преобразуют выходной код шифратора к требуемой разрядности.

Дешифратор (Decd): Формирует сигналы подтверждения нажатия конкретных клавиш.

Сигналы управления:

- ok — подтверждение ввода цифры;
- ok_f — флаг подтверждения;
- new / new_vod — сигналы начала новой игры или нового ввода.

6.3.2. Шестнадцатеричные индикаторы.



Скриншот 22. Ввод игрока.

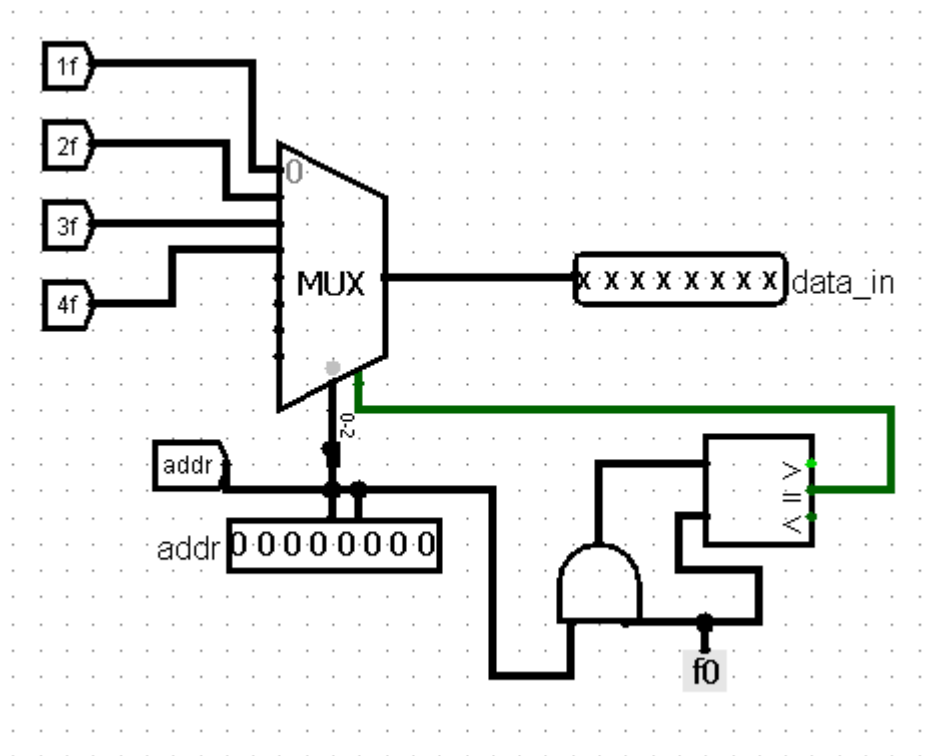
Отображают 4-значное число (код), который ввёл игрок.

Состав и элементы:

- Четыре шестнадцатеричных индикатора: Выходные устройства, расположенные в верхней части блока. Каждый индикатор отображает одну цифру.
- Входы (D): Подключены к общей шине данных. Через эту шину процессор или логика ввода записывает значение цифры.
- Выходы (Q): Подключены непосредственно к соответствующим индикаторам.
- Управление (en/clock): Сигналы разрешения записи и тактирования.

Когда игрок вводит число, соответствующие данные попадают в регистры 1f, 2f, 3f, 4f. Регистры удерживают эти значения. Выходы регистров постоянно управляют индикаторами дисплеев, обеспечивая стабильное свечение цифр на экране. Этот блок работает в связке с блоком ввода: то, что игрок набрал на кнопках, сохраняется в этих регистрах и сразу отображается на индикаторах, давая визуальную обратную связь.

6.3.3. Мультиплексор ввода данных.



Скриншот 23. Реализация мультиплексора.

Выбор источника данных для шины data_in. При вводе цифры пользователем она сохраняется в соответствующий регистр (1f-4f). Процессор может прочитать содержимое любого регистра, обратившись к его адресу.

Состав и функционирование:

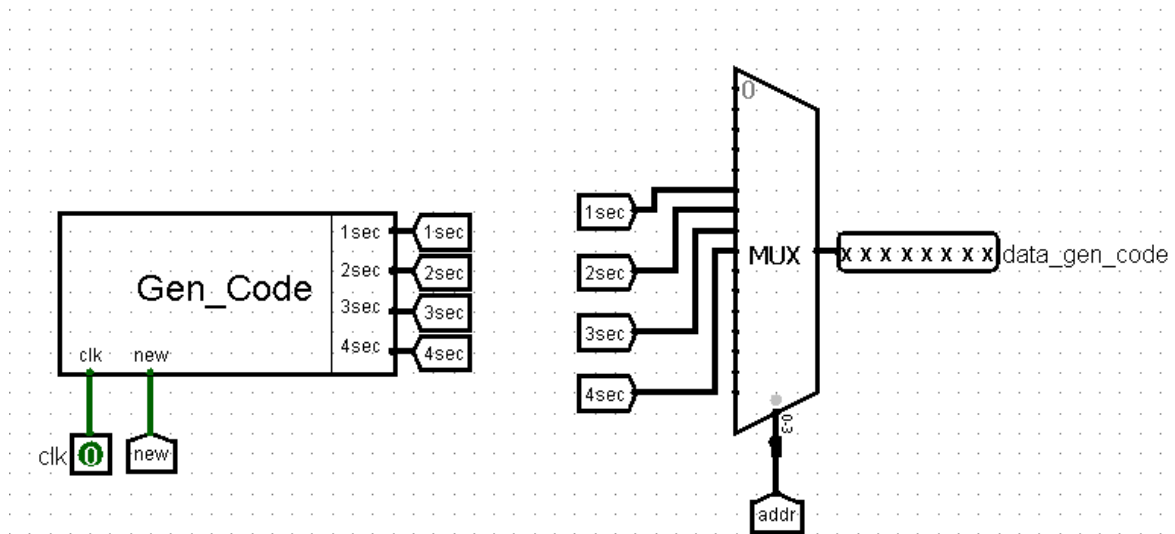
Мультиплексор (MUX): Коммутирует один из четырех входов (1f, 2f, 3f, 4f) на выход data_in.

Управление: Адресный вход addr определяет, какой из регистров подключается к шине данных.

Регистры (1f-4f): Хранят введенные пользователем цифры.

Логические элементы: Обеспечивают синхронизацию записи в регистры по сигналу ok.

6.3.4. Блок генератора кода.



Скриншот 24. Генератор секретного кода Gen_Code

Состав и функционирование:

Блок Gen_Code: Генерирует четыре случайные цифры (1sec, 2sec, 3sec, 4sec).

Управляющие сигналы:

- clk — тактовый сигнал для генерации псевдослучайной последовательности;
- new — сигнал начала новой игры.

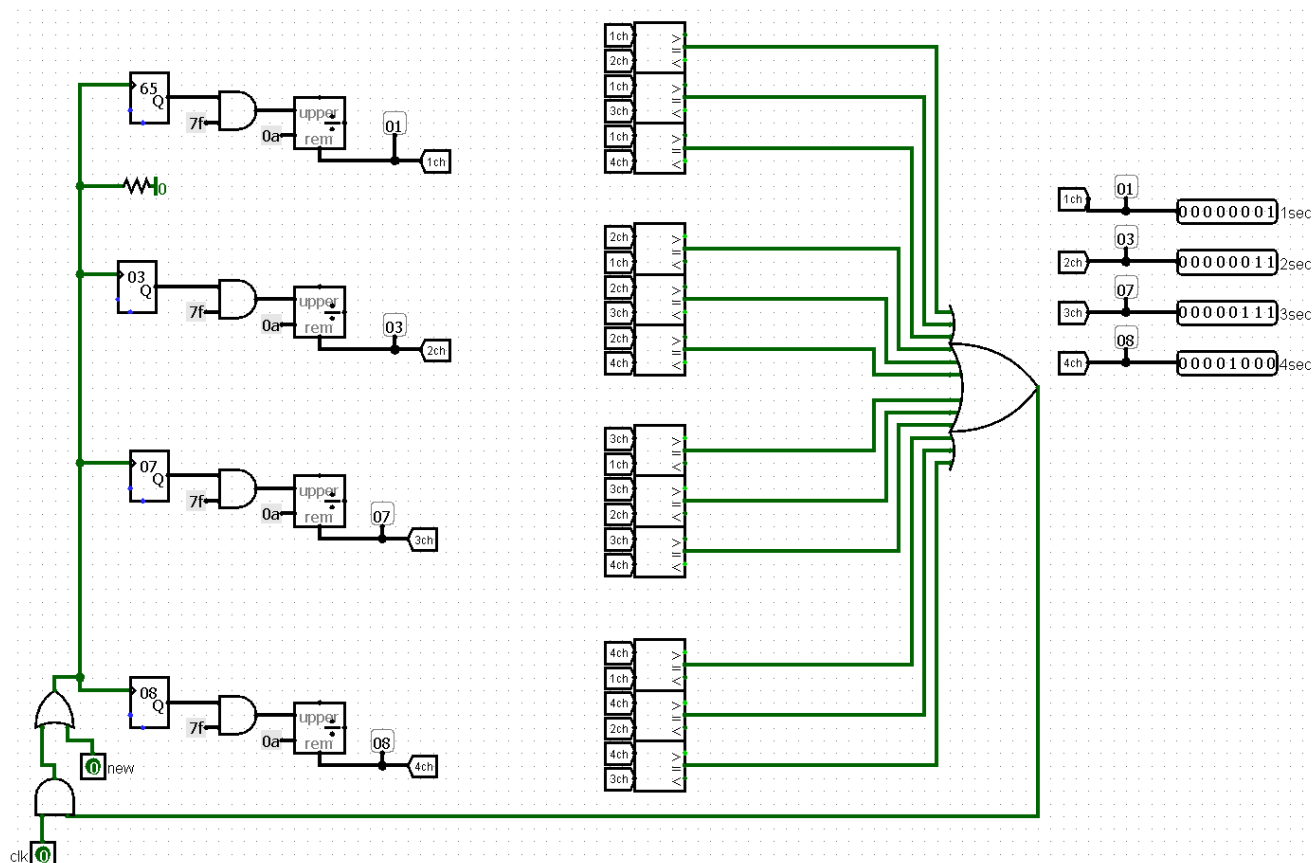
Мультиплексор: Выбирает одну из четырех сгенерированных цифр для передачи на шину data_gen_code.

Управление адресом: Вход addr определяет, какой разряд кода считывается.

При старте новой игры генератор формирует секретный код из 4 цифр. Процессор может прочесть любую цифру кода, обратившись к соответствующему адресу через мультиплексор.

6.4. GEN_CODE

На следующей схеме происходит генерация 4-значного кода в котором гарантируется, что все цифры различны.



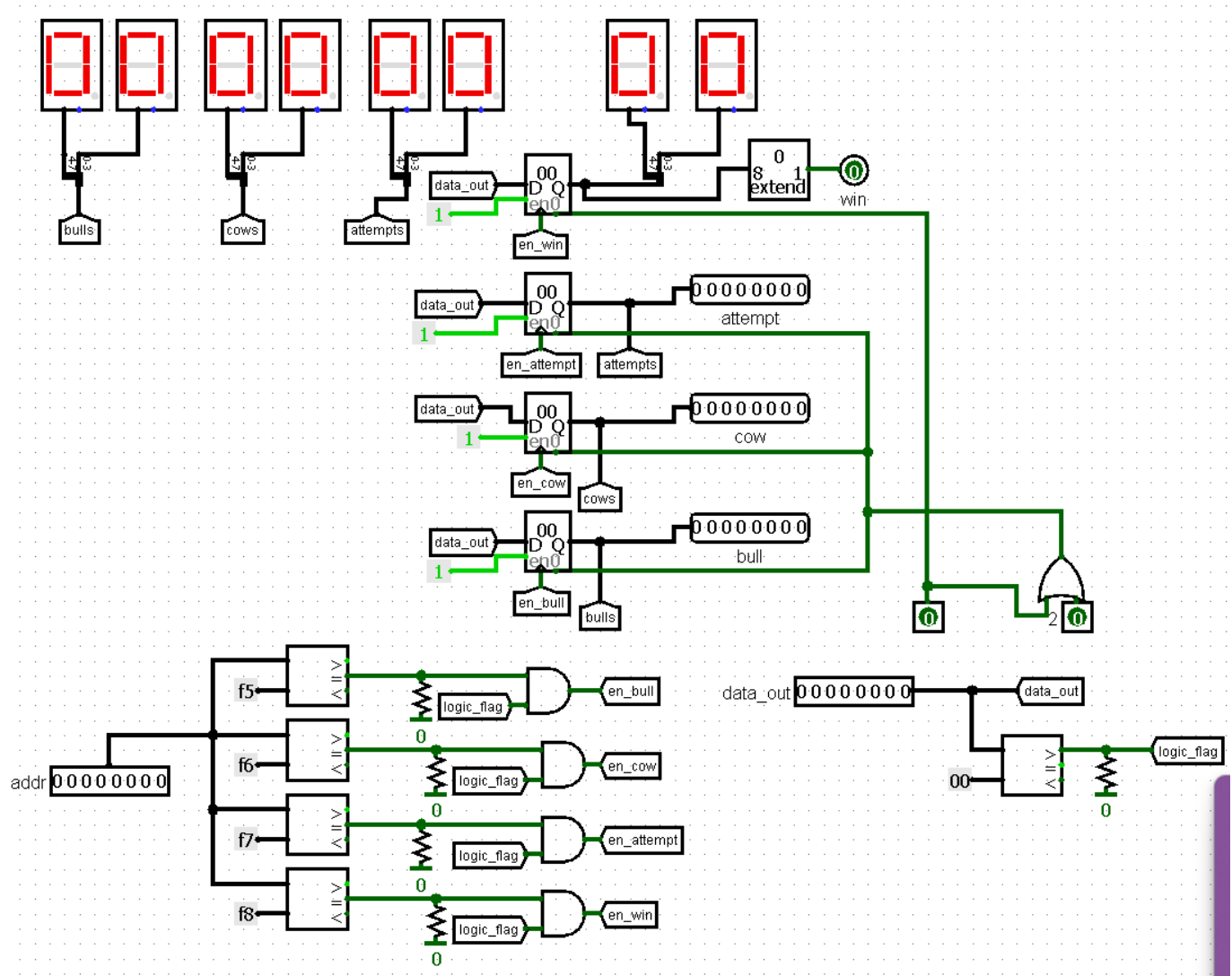
Скриншот 25. Схема «GEN_CODE».

Процесс запускается сигналом new, который сбрасывает и инициализирует четыре параллельных генератора случайных чисел, расположенных в левой части схемы.

Процесс работы выглядит следующим образом: четыре параллельных генератора случайных чисел (каналы 1ch, 2ch, 3ch, 4ch), число через маску приводится к положительному виду и берёт остаток от деления на 10 – таким образом, из генератора числа мы получаем цифру, которые одновременно поступают на входы группы компараторов; каждый компаратор выполняет операцию сравнения двух разных каналов (например, первый разряд сравнивается со вторым, первый с третьим, второй с четвертым и т.д.), выявляя наличие одинаковых цифр в сгенерированном наборе. Выходные сигналы всех компараторов собираются на общий логический элемент **ИЛИ**, который функционирует как детектор дубликатов: если хотя бы одна пара цифр совпадает, компараторы выдают единицу, которая через элемент ИЛИ блокирует фиксацию результата, заставляя генераторы продолжать перебор значений. Только в тот момент, когда все компараторы подтверждают неравенство каждого разряда (сигнал на выходе ИЛИ становится нулевым), сформированная комбинация признается валидной и записывается в выходные регистры, передавая процессору готовый код, в котором гарантированно отсутствуют повторяющиеся цифры.

6.5. OUTPUT_IO.

Реализация порта вывода для отображения результатов проверки хода игрока (количества быков и коров) с использованием метода отображения ввода-вывода на память (Memory-Mapped I/O).

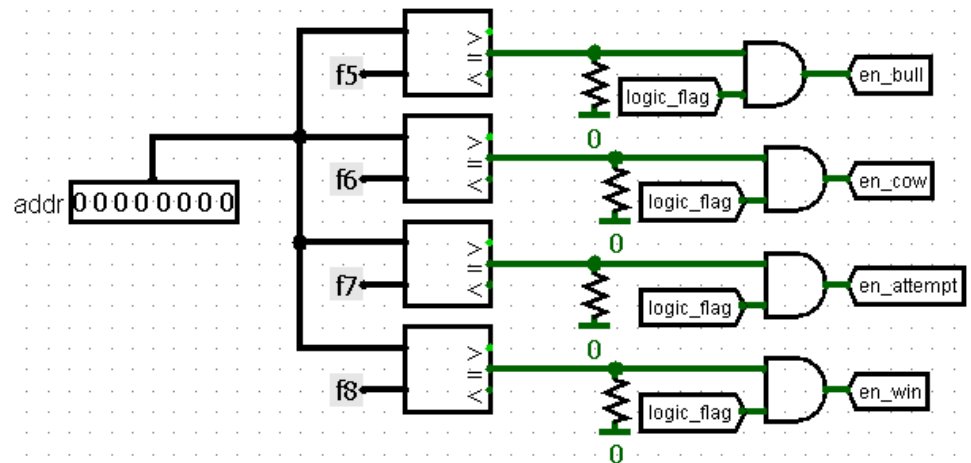


Скриншот 26. Схема «OUTPUT_IO».

6.5.1. Дешифратор адреса (Address Decoder).

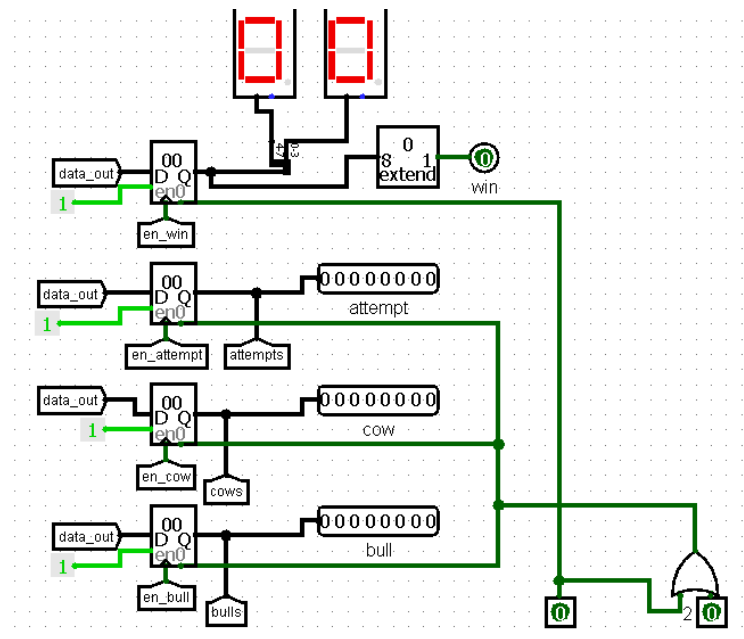
Четыре компаратора (f5, f6, f7, f8). Определение момента обращения процессора к порту вывода путём сравнения текущего значения адресной шины addr с предустановленными адресами портов вывода.

Каждый компаратор непрерывно сравнивает входной адрес с константой (отвечающей за адрес). При совпадении адресов компаратор выдаёт логическую «1» на свой выход. Если процессор обращается к любому из адресов диапазона f5–f8, соответствующий компаратор активируется, сигнализируя о выборе данного устройства вывода.



Скриншот 27. Дешифратор.

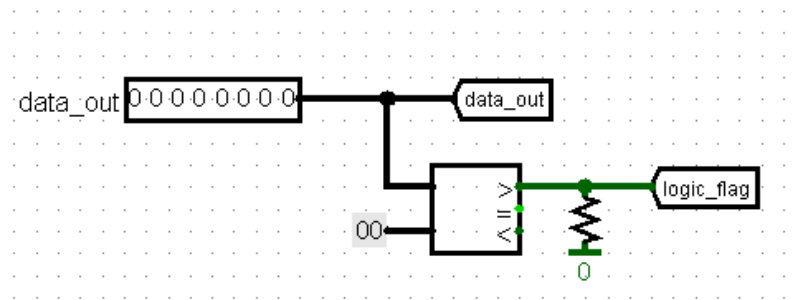
6.5.2. Учет статистики и индикация результата.



Скриншот 28. Блок учета статистики и индикации результата.

Он отвечает за хранение текущего счета игры (количество быков, коров и попыток) и отображение статуса победы.

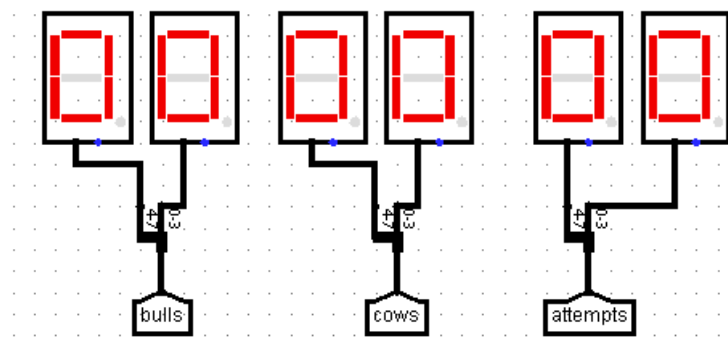
6.5.3. Детекция нулевого значения.



Скриншот 29. Блок детекции нулевого значения.

Логический флаг. Если данные больше нуля (для устранения ошибок при передаче данных).

6.5.4. Визуальная индикация игровой статистики.

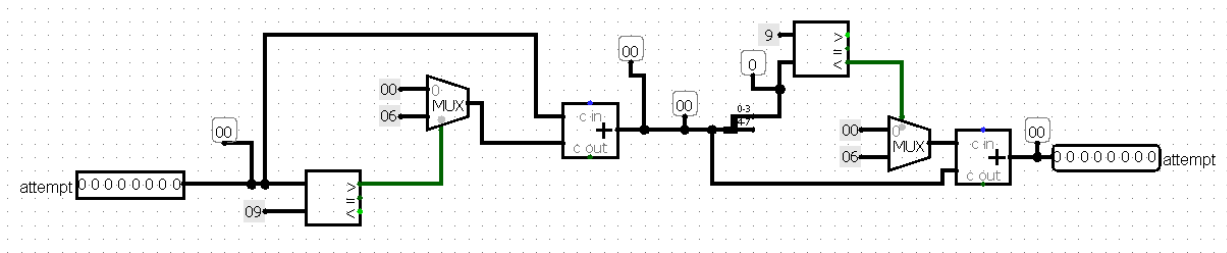


Скриншот 30. Блок визуализации.

Обеспечивает вывод текущих значений быков, коровы и попытки. Отображает прогресс игры для игрока, преобразуя двоичные данные регистров в десятичные числа на табло.

- 6 шестнадцатеричных индикаторов, которые сгруппированы по парам.
- Разветвитель для вывода десятков и единиц.

6.6. bcd.

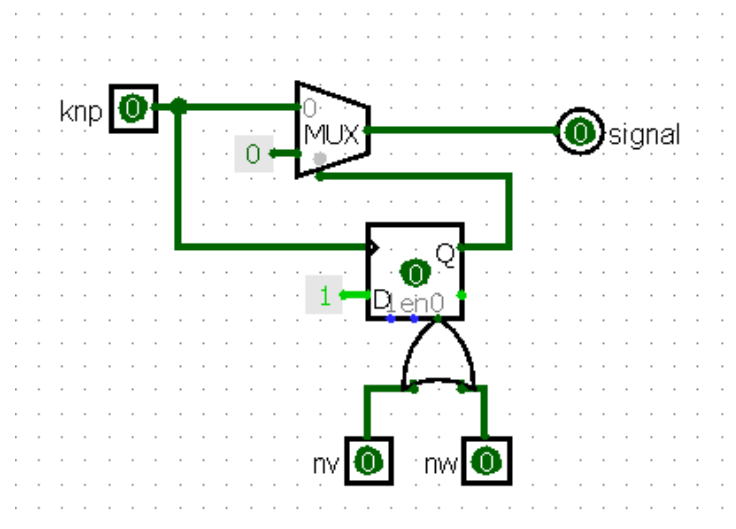


Скриншот 31. Схема «bcd».

Данная подсхема реализует BCD-коррекцию (двоично-десятичное кодирование) для счётчика попыток. Схема инкрементирует счётчик попыток и применяет коррекцию. При достижении значения «9» следующая инкрементация должна дать 10, а не «0A». Для этого добавляется константа «6», которая «перескакивает» через недопустимые BCD – коды (A-F). Результат корректно отображается как десятичное число на индикаторах.

Ранее BCD использовалась в блоке отображения на шестнадцатеричных индикаторах(см. пункт 4.1.), в блоке OUTPUT_IO (см. пункт 4.5.) и в блоке статистики (см. пункт 4.5.2.).

6.7. block.



Скриншот 32. Схема «block».

Этот фрагмент представляет собой синхронную защёлку события нажатия кнопки. Он гарантирует, что каждая из 10 цифр (0-9) может быть введена только один раз при формировании 4-значного кода. Для каждой цифры в схеме стоит свой такой блок, и их выходы объединяются логикой, которая не даёт завершить ввод, пока не выбрано 4 различные цифры. При передаче сигнала с кнопки d-триггер, работающий по заднему фронту, обрабатывает сигналы и после нажатия кнопки d-триггер ставит состояние константы единицу, что вместе с мультиплексором работает как защёлка.

Входы:

- 1 – битный контакт kpr – нажатие кнопки.
- 1 битный контакт pv – отщёлкивание при новом вводе.
- 1 битный контакт pw – отщёлкивание при новой игре.

Выходы:

- 1 битный контакт signal – итоговый сигнал «цифра занята».

6.8. assembler.

Программа на языке ассемблер, которая является ядром игры. Она работает с полученным кодом от пользователя подсчитывая значения быков и коров.

Программа начинается с директив asect, которые резервируют ячейки в памяти данных под игровые переменные:

0xE5–0xE8: sec0...sec3 — ячейки для хранения загаданного секретного кода (по одной цифре в каждой).

0xF0–0xF8:

- in0...in3 — ячейки, по которым аппаратная часть читает цифры, введенные игроком.
- ok — флаг подтверждения хода (1 = игрок завершил ввод, 0 = ожидание).
- bulls, cows, tries — счетчики результатов и попыток.
- win — флаг окончания игры (1 = победа, 0 = игра продолжается).

Основной код программы размещается с адреса 0x00.

```

1  asect 0xE5
2  sec0:  ds 1
3  sec1:  ds 1
4  sec2:  ds 1
5  sec3:  ds 1
6
7  asect 0xF0
8  in0:   ds 1
9  in1:   ds 1
10 in2:   ds 1
11 in3:   ds 1
12 ok:    ds 1
13 bulls: ds 1
14 cows:  ds 1
15 tries: ds 1
16 win:   ds 1
17
18 asect 0x00

```

Скриншот 33. Директивы.

Инициализация (main)

При старте процессор обнуляет все счетчики (bulls, cows, tries) и флаг win, подготавливая систему к новой игре.

Главный игровой цикл (loop)

Цикл начинается с проверки флага win. Если флаг не равен нулю (tst r1 / is nz), управление передается на метку finish, где выполняется команда halt (остановка процессора). Это гарантирует выход из программы при победе.

Ожидание ввода (wait_ok)

Процессор переходит в цикл чтения флага ok. Он читает значение из памяти (ld r0, r1) и сравнивает его с единицей. Если значение равно 1 (cmp r1, r3 / is eq), значит игрок ввел 4 цифры и подтвердил ход, происходит переход к блоку ready. В противном случае цикл повторяется, процессор ожидает действия пользователя.

Подсчет (ready)

Быков: Перед анализом счетчики bulls и cows обнуляются. Регистр r2 используется как временный счётчик совпадений. Логика «быков» реализована прямым посимвольным сравнением на одинаковых позициях:

- in0 сравнивается с sec0
- in1 сравнивается с sec1
- in2 сравнивается с sec2
- in3 сравнивается с sec3

Для каждой пары значения загружаются в регистры r1 и r3, сравниваются (cmp), и при равенстве (is eq) счетчик r2 увеличивается на 1 (inc r2). Итоговое количество совпавших позиций сохраняется в переменную bulls.

Проверка условия победы

Счетчик быков (r2) сравнивается с константой 4. Если они равны, в ячейку win записывается 1. На следующем витке цикла loop программа обнаружит этот флаг и завершит работу.

Подсчет «Коров»

Счетчик r2 сбрасывается. Логика «коров» реализована через явные перекрестные сравнения, исключая уже учтенные «быки»:

- Цифра in0 сравнивается с sec1, sec2, sec3

- Цифра in1 сравнивается с sec0, sec2, sec3
- Цифра in2 сравнивается с sec0, sec1, sec3
- Цифра in3 сравнивается с sec0, sec1, sec2

Каждое найденное совпадение увеличивает r2. Таким образом, цифра, которая уже стала «быком», не проверяется повторно, что исключает двойной учет. Результат сохраняется в cows.

Завершение хода и возврат в цикл

- Увеличивается счетчик попыток tries.
- Флаг ok сбрасывается в 0, чтобы система снова перешла в режим ожидания (wait_ok).
- Выполняется безусловный переход br loop, запуская новый раунд проверки.

7 ЗАКЛЮЧЕНИЕ

В ходе выполнения данного проекта была разработана игра на логику «Быки и коровы» на основе процессора Cdm-8 в среде моделирования Logisim с ассемблером. Были спроектированы схемы процессора, генератор случайного кода с проверкой уникальности цифр, система ввода данных с клавиатуры и вывода информации на индикаторы, а также написана программа на языке ассемблер для подсчёта быков и коров.

В процессе разработки возникли трудности с организацией корректной работы шины данных и предотвращением конфликтом при обращении к разным блокам памяти. Также потребовалась отладка алгоритма генерации, чтобы исключить повтор цифр. Однако все проблемы были решены и в данный момент система работает стабильно.

В ходе выполнения проекта были закреплены теоретические знания по архитектуре ЭВМ, получены практические навыки проектирования цифровых схем и программирования на ассемблере.

В дальнейшем можно усовершенствовать игру. Например, добавить таблицу рекордов и увеличить количество попыток.

8 ИСТОЧНИКИ

Computing platforms, A.Shafarenko and S.P.Hunt.

Э. ТАНЕНБАУМ АРХИТЕКТУРА КОМПЬЮТЕРА.